Journo — Trình Bày Dự Án Fullstack (Tiếng Việt)

Phần 2: Giải Thích Sâu Từng Thành Phần Kỹ Thuật

> Tiếp theo từ Phần 1 — Tổng quan & Kiến trúc

---

3. GIẢI THÍCH SÂU TỪNG THÀNH PHẦN

3.1 AUTH FLOW — Laravel Sanctum Token Authentication

WHY: Tại sao dùng Sanctum thay vì tự viết JWT?

Tôi chọn Laravel Sanctum vì dự án này là SPA (Single Page Application) cần API token authentication. Sanctum cung cấp sẵn:

- Bảng `personal_access_tokens` để lưu và revoke token
- Middleware `auth:sanctum` tích hợp vào routing
- Hàm `createToken()` với mã hóa an toàn

So với tự viết JWT: JWT là stateless — server không lưu token, không thể revoke trước khi hết hạn. Sanctum lưu hash của token trong database, nên có thể logout thật sự (xóa token). Đây là tradeoff quan trọng tôi đã cân nhắc.

HOW: Cơ chế hoạt động của Sanctum

```
// AuthController.php — Login
public function login(Request $request)
{
    $user = User::where('email', $request->email)->first();
    // Truy vấn DB tìm user theo email

    if (!$user) {
        return response()->json(['message' => 'Email not found'], 404);
        // Trả 404 thay vì 401 — đây là design decision:
        // giúp UX nhưng cũng leak thông tin email tồn tại hay không
    }

    $passwordIsCorrect = Hash::check($request->password, $user->password);
    // Hash::check() dùng bcrypt verify:
    // - Lấy salt từ hash đã lưu trong DB
    // - Hash plaintext password với salt đó
    // - So sánh kết quả với stored hash
    // → Không thể reverse-engineer password từ hash

    if (!$passwordIsCorrect) {
        return response()->json(['message' => 'Wrong password'], 401);
    }
```

```php
    $token = $user->createToken('auth_token')->plainTextToken;
    // createToken() của Sanctum:
    // 1. Tạo random 40-character token string
    // 2. Hash token đó bằng SHA-256
    // 3. Lưu hash vào bảng personal_access_tokens
    // 4. Trả về plaintext: format "id|randomString"
    // Ví dụ: "3|Kx9mN2pQrT..." (id=3, phần sau là random)

    return response()->json(['user' => $user, 'token' => $token], 200);
}
```

```php
// AuthController.php — Logout
public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();
    // currentAccessToken() lấy token đang được dùng trong request này
    // delete() xóa record khỏi personal_access_tokens
    // → Token hết hiệu lực ngay lập tức, không cần chờ hết hạn
}
```

```php
// User.php Model
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens;
    // Trait này add các method: createToken(), tokens(), currentAccessToken()
    // Đây là design pattern Mixin/Trait của PHP —
    // thêm behavior mà không cần kế thừa
}
```

```javascript
// frontend/src/api/axios.js — Token Attachment
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  // Đọc token từ localStorage mỗi request
  // Không cache vào biến JS để tránh stale value sau logout

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
    // Format chuẩn RFC 6750: "Bearer <token>"
    // Sanctum middleware đọc header này để xác thực
  }
  return config;
  // PHẢI return config, không thì request bị block
});
```

Luồng token verification phía backend:

```
Request với header: Authorization: Bearer 3|Kx9mN2pQrT...
        │
        ▼
Sanctum middleware tách: id=3, token=Kx9mN2pQrT...
        │
        ▼
Hash token bằng SHA-256 → tìm trong personal_access_tokens WHERE id=3
        │
        ▼
So sánh hash → nếu khớp: load user, bind vào $request
        │
        ▼
Controller nhận được $request->user() đã có giá trị
```

TRADEOFF

	Sanctum (stateful token)	JWT (stateless)
Logout thật sự	Có (xóa DB record)	Không (phải chờ hết hạn)
Revoke từng token	Có	Không
Scalability	Mỗi request cần hit DB	Không cần DB check
Phù hợp với	SPA, mobile app cần logout	Microservices, scale lớn

---

3.2 TOKEN STORAGE — Tại sao lưu localStorage

HOW: Cách tôi lưu và đọc token

```jsx
// Login.jsx — Lưu token sau khi login thành công
localStorage.setItem("token", token);
// localStorage.setItem(key, value) — persist qua reload, tab khác

// Navbar.jsx — Đọc token để quyết định hiện UI nào
const token = localStorage.getItem("token");
const isLoggedIn = token !== null;

// Logout
localStorage.removeItem("token");
window.location.href = "/login";
```

WHY: Tại sao không dùng httpOnly cookie?

Tôi chọn localStorage vì đơn giản nhất cho SPA development và không cần setup CSRF token. Tuy nhiên, tôi nhận thức rõ đây là **điểm yếu bảo mật**: nếu có XSS attack, JavaScript của attacker có thể đọc được token từ

localStorage.

Với httpOnly cookie: JavaScript không đọc được cookie → XSS không lấy được token. Nhưng cần xử lý CORS `credentials: true` và CSRF protection phức tạp hơn.

Kết luận thành thật: Tôi sẽ migrate sang httpOnly cookie trong production.

3.3 STATE MANAGEMENT — Local State thay vì Context/Redux

WHY: Tại sao không dùng Context API hay Redux?

Tôi quyết định **không dùng Context API hay Redux** vì scope của project chưa cần. Phân tích:

- **Auth state:** Token lưu trong localStorage → mọi component đọc trực tiếp từ `localStorage.getItem("token")`. Không cần share qua Context.
- **Post data, comment data:** Chỉ dùng ở 1-2 component → không cần global state.
- **Form state:** Chỉ cần trong form component đó → `useState` là đủ.

Nếu dùng Redux cho project này sẽ là **over-engineering** — thêm boilerplate (actions, reducers, store) mà không giải quyết vấn đề thực tế nào.

HOW: Pattern local state tôi dùng nhất quán

```
// Home.jsx — Pattern điển hình
const [posts, setPosts] = useState([]);      // data
const [loading, setLoading] = useState(true); // loading state
const [error, setError] = useState(null);    // error state

useEffect(() => {
  fetchPosts();
}, []); // [] = chỉ chạy 1 lần khi component mount

const fetchPosts = async () => {
  setLoading(true);   // bắt đầu: bật loading
  setError(null);     // reset error từ lần trước

  try {
    const res = await api.get("/posts");
    setPosts(res.data.data); // data.data vì Laravel paginate wrap trong { data:
[...] }
  } catch (err) {
    setError("Failed to load posts"); // user-friendly error message
  } finally {
    setLoading(false); // luôn tắt loading dù thành công hay thất bại
  }
};
```

Pattern 3-state (loading/error/data) này tôi áp dụng nhất quán:

- `loading=true` → hiện spinner/skeleton

- `error!=null` → hiện error message
- cả hai false → hiện data

TRADEOFF: Server State vs Client State

Trong project này tôi chưa phân biệt rõ server state và client state. Tất cả đều là `useState`. Trong production, tôi sẽ dùng **React Query** hoặc **TanStack Query** để:

- Cache server data
- Auto-refetch khi stale
- Deduplicate concurrent requests
- Background sync

3.4 ROUTING — React Router DOM v7

HOW: Cấu hình routes

```
// App.jsx
export default function App() {
  return (
    <BrowserRouter>
      {/* Navbar render ở mọi route */}
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
        <Route path="/posts/:id" element={<PostDetail />} />
        <Route path="/create-post" element={<CreatePost />} />
        <Route path="/posts/:id/edit" element={<EditPost />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Điểm quan trọng: Tôi **không có Protected Route component**. Thay vào đó, backend xử lý authorization — nếu user truy cập `/create-post` mà không có token, họ thấy form nhưng khi submit sẽ nhận 401 từ backend.

Điểm yếu: UX không tốt — nên redirect về `/login` ngay khi chưa auth, trước khi user điền form. Fix bằng cách tạo `PrivateRoute` component.

HOW: Đọc URL params

```
// PostDetail.jsx và EditPost.jsx
import { useParams } from "react-router-dom";

const { id } = useParams();
```

```
// useParams() trả về object chứa tất cả :param trong route
// Route: /posts/:id → useParams() = { id: "42" }
// Lưu ý: id là string, không phải number
```

HOW: Navigation

```
// Tôi dùng window.location.href thay vì useNavigate()
window.location.href = "/";

// Lý do thực tế: useNavigate() cần component mount
// window.location.href = full page reload → đảm bảo token được đọc lại
// Nhược điểm: mất state React, kém hơn SPA navigation
```

3.5 API DESIGN — RESTful Conventions

HOW: Naming và HTTP Methods tôi dùng

```
GET    /api/posts              → index()  — lấy danh sách (idempotent)
GET    /api/posts/{id}         → show()   — lấy 1 bài (idempotent)
POST   /api/posts              → store()  — tạo mới (không idempotent)
PUT    /api/posts/{id}         → update() — cập nhật toàn bộ
DELETE /api/posts/{id}         → destroy() — xóa

POST   /api/register           → tạo user mới
POST   /api/login              → lấy token (action, không phải resource)
POST   /api/logout             → revoke token

GET    /api/posts/{id}/comments → nested resource — comments của 1 post
POST   /api/posts/{id}/comments → tạo comment cho 1 post
```

Tôi dùng `PUT` thay vì `PATCH` cho update post. Sự khác biệt:

- `PUT`: thay thế toàn bộ resource (phải gửi tất cả fields)
- `PATCH`: partial update (chỉ gửi field cần thay đổi)

Trong code thực tế tôi xử lý partial update (`if ($request->title)`) nhưng dùng method `PUT` — đây là inconsistency nên sửa thành `PATCH`.

HOW: Response format

```
// Thành công — tạo mới
return response()->json($post, 201);  // 201 Created

// Thành công — truy vấn
return response()->json($posts, 200);  // 200 OK (hoặc bỏ qua 200 vì default)
```

```
// Lỗi client
return response()->json(['message' => 'Post not found'], 404);

// Lỗi permission
return response()->json(['message' => 'You do not have permission...'], 403);

// Laravel Pagination format (PostController::index)
// Tự động wrap thành:
// { data: [...], current_page: 1, last_page: 3, per_page: 10, total: 25 }
```

Frontend đọc paginated response:

```
const res = await api.get("/posts");
setPosts(res.data.data);
// res.data = toàn bộ pagination object
// res.data.data = mảng posts thực sự
```

3.6 DATABASE LAYER — Eloquent ORM

WHY: Tại sao dùng Eloquent thay vì raw SQL?

Tôi chọn Eloquent vì:

1. Tự động escape input → tránh SQL injection
2. Relationships dễ define và query
3. Migration system giúp schema version control
4. Thời gian phát triển nhanh hơn

HOW: Eager Loading chống N+1

```
// PostController::index() — ĐÚNG
$posts = Post::with('author', 'category', 'tags')
    ->where('status', 'published')
    ->paginate(10);

// with('author', 'category', 'tags') = Eager Loading
// Laravel thực hiện 4 queries thay vì N+1:
// Query 1: SELECT * FROM posts WHERE status='published' LIMIT 10
// Query 2: SELECT * FROM users WHERE id IN (1,2,3,...)   ← author
// Query 3: SELECT * FROM categories WHERE id IN (...)    ← category
// Query 4: SELECT * FROM tags JOIN post_tag WHERE post_id IN (...)  ← tags
```

Nếu không có eager loading — N+1 problem:

```php
// SAI — N+1
$posts = Post::where('status', 'published')->paginate(10);
foreach ($posts as $post) {
    echo $post->author->name; // Mỗi lần access: thêm 1 query
    // 10 posts = 10 queries thêm cho author → tổng 11 queries
}
```

HOW: Relationship Definitions

```php
// Post.php Model
public function author()
{
    return $this->belongsTo(User::class, 'user_id');
    // belongsTo: Post thuộc về User
    // 'user_id' là foreign key trong posts table
    // alias 'author' thay vì 'user' để rõ ràng hơn trong context
}

public function tags()
{
    return $this->belongsToMany(Tag::class, 'post_tag');
    // N-N relationship qua junction table post_tag
    // Laravel tự handle JOIN
}

public function comments()
{
    return $this->hasMany(Comment::class);
    // 1-N: 1 post có nhiều comments
    // auto-detect foreign key: post_id trong comments table
}
```

```php
// Comment.php — Self-referencing relationship
public function replies()
{
    return $this->hasMany(Comment::class, 'parent_id');
    // Comment có nhiều replies (cũng là Comment)
    // 'parent_id' là FK thay vì default 'comment_id'
}
```

HOW: Authorization check ở Controller

```php
// PostController::update()
$post = Post::find($id);
```

```
if ($post->user_id !== $request->user()->id) {
    return response()->json(['message' => '...'], 403);
    // Kiểm tra: author của post có phải user đang request không?
    // $request->user()->id: lấy từ Sanctum, KHÔNG thể giả mạo
    // $post->user_id: lấy từ DB
    // Nếu không match: 403 Forbidden
}
```

3.7 SECURITY

3.7.1 Password Hashing

```
// AuthController::register()
$user->password = bcrypt($request->password);
// bcrypt() = PHP wrapper cho password_hash($password, PASSWORD_BCRYPT)

// .env
BCRYPT_ROUNDS=12
// Cost factor 12: 2^12 = 4096 iterations
// Cân bằng: đủ chậm để brute-force khó, đủ nhanh để login responsive
// Tại sao không MD5/SHA256?
// - MD5/SHA256 quá nhanh: GPU có thể thử hàng tỷ hash/giây
// - Không có salt tự động → rainbow table attack
// - Bcrypt: built-in salt, adaptive cost factor
```

```
// AuthController::login() — Verify password
$passwordIsCorrect = Hash::check($request->password, $user->password);
// Hash::check() extract salt từ stored hash,
// hash lại input với same salt, so sánh
// → timing-safe comparison (tránh timing attack)
```

3.7.2 SQL Injection Prevention

```
// Eloquent tự dùng PDO prepared statements
$user = User::where('email', $request->email)->first();
// Thực ra là: SELECT * FROM users WHERE email = ? [binding: email value]
// Input KHÔNG bao giờ được ghép trực tiếp vào SQL string
// → SQL injection không thể xảy ra qua Eloquent queries

// Laravel validation thêm một lớp nữa:
$request->validate(['email' => 'required|email']);
// Validate trước khi đưa vào query
```

3.7.3 Authorization (Ai được làm gì)

```
// PostController — Pattern tôi dùng nhất quán:
// 1. Tìm resource
$post = Post::find($id);

// 2. Check tồn tại
if (!$post) { return response()->json([...], 404); }

// 3. Check quyền
if ($post->user_id !== $request->user()->id) {
    return response()->json([...], 403);
}

// 4. Thực hiện action
$post->delete();
```

Lưu ý: Tôi chưa dùng Laravel Policies hay Gates — đây là cải thiện có thể làm sau.

3.7.4 CORS Configuration

```
// bootstrap/app.php — Laravel tự handle CORS
// Vite dev server chạy trên port 5173
// Backend chạy trên port 8000
// → Cần CORS để browser cho phép cross-origin request

// config/cors.php (Laravel default):
// 'allowed_origins' => ['*']  ← Development: cho phép all origins
// Production cần restrict: ['https://journo.com']
```

Frontend Axios không cần config CORS — CORS là policy của server, không phải client. Client chỉ nhận response header `Access-Control-Allow-Origin` từ server.

3.8 DESIGN PATTERNS TRONG CODE

Pattern 1: Repository Pattern (qua Eloquent)

3 dấu hiệu trong code:

1. Controller không viết SQL trực tiếp — chỉ gọi `Post::find()`, `Post::where()`
2. Data access logic tập trung ở Model class
3. Nếu đổi DB engine, chỉ cần đổi Model, không đổi Controller

```
// Controller chỉ nói "lấy cái gì", không quan tâm "lấy như thế nào"
$post = Post::with('author', 'category', 'tags')->find($id);
```

WHY: Tách data access khỏi business logic
TRADEOFF: Tôi chưa tạo Repository class thực sự — Eloquent đóng vai repository nhưng lẫn với Model. Nếu cần test unit không cần DB, cần tách ra hơn.

Pattern 2: Middleware Chain

3 dấu hiệu trong code:

1. `Route::middleware('auth:sanctum')->group(...)` bọc nhóm routes
2. Middleware chạy trước controller — không cần check auth trong controller
3. `bootstrap/app.php` cấu hình exception middleware

```
// routes/api.php
Route::middleware('auth:sanctum')->group(function () {
    // Mọi route trong đây đều đi qua Sanctum middleware trước
    Route::post('/posts', [PostController::class, 'store']);
    // Controller không cần if (!auth) — middleware đã handle
});
```

WHY: Single Responsibility — middleware lo auth, controller lo business logic
TRADEOFF: Middleware chain cứng — khó test từng bước riêng lẻ

Pattern 3: Active Record (Eloquent Model)

3 dấu hiệu trong code:

1. Model object biết cách lưu chính nó: `$post->save()`
2. Model biết cách xóa chính nó: `$post->delete()`
3. Model biết relationships của mình: `$post->author`, `$post->comments`

```
// PostController::store()
$post = new Post();              // tạo instance
$post->title = $request->title; // set attributes
$post->save();                   // model tự INSERT vào DB

// Model biết nó liên kết với bảng posts, biết primary key là id
// Không cần viết SQL INSERT
```

WHY: Developer experience tốt, nhanh develop
TRADEOFF: Model làm quá nhiều việc (data + logic) — vi phạm Single Responsibility nếu thêm nhiều business logic vào model

Pattern 4: Interceptor Pattern (Axios)

3 dấu hiệu trong code:

1. Logic gắn token tách khỏi từng API call cụ thể

2. Tất cả requests đều đi qua interceptor trước khi gửi
3. Thay đổi auth logic chỉ ở 1 chỗ

```
// api/axios.js — Interceptor setup (1 lần duy nhất)
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Mọi page dùng api — không cần tự gắn token
// Home.jsx:      api.get("/posts")     ← tự động có Bearer
// CreatePost.jsx: api.post("/posts")   ← tự động có Bearer
// EditPost.jsx:  api.put("/posts/1")   ← tự động có Bearer
```

WHY: DRY (Don't Repeat Yourself) — không lặp token logic ở mọi component
TRADEOFF: Nếu một số request không cần token (hoặc cần khác), phải handle exception trong interceptor

---

3.9 QUYẾT ĐỊNH THIẾT KẾ KHÔNG HIỂN NHIÊN

1. `window.location.href` thay vì `useNavigate()`

```
// Login.jsx, Register.jsx, CreatePost.jsx
window.location.href = "/";  // Không dùng navigate("/")
```

Lý do: Sau login, Navbar cần re-read token từ localStorage để hiện nút "Logout" thay vì "Login". Với `useNavigate()`, React chỉ re-render route — Navbar component không unmount/remount → không re-read localStorage.

`window.location.href` trigger full page reload → Navbar mount lại → đọc token mới → hiện đúng UI.

Cái giá: Mất đi SPA seamless navigation. Fix tốt hơn: dùng Context/state để share auth status.

2. `user_id` trong Post được lấy từ `$request->user()` không phải từ request body

```
$post->user_id = $request->user()->id;
// KHÔNG phải: $post->user_id = $request->user_id;
```

Lý do bảo mật: Nếu lấy `user_id` từ request body, client có thể giả mạo và tạo post thay mặt người khác. `$request->user()` là user đã được Sanctum xác thực — không thể giả mạo.

3. `is_approved = true` hardcode trong CommentController

```
// CommentController::store()
$comment->is_approved = true;  // Hardcode approve
```

Lý do: Chưa implement moderation flow. Schema đã thiết kế sẵn `is_approved` field cho tương lai. Hiện tại hardcode `true` để comments hiện ngay, không cần admin approve.

Vấn đề: Không có moderation → spam risk. Cần implement approval queue.

4. Slug auto-generate nhưng không handle duplicate

```
$post->slug = Str::slug($request->title);
// "My First Post" → "my-first-post"
```

Vấn đề: `posts.slug` có `UNIQUE` constraint. Nếu 2 bài có cùng title → duplicate slug → database error không được handle gracefully.

Fix: Append timestamp hoặc random suffix: `Str::slug($title) . '-' . time()`

5. Comment fetch tách biệt với post fetch

```
// PostDetail.jsx
useEffect(() => {
  fetchPost();     // 2 API calls riêng biệt
  fetchComments(); // thay vì 1 call duy nhất
}, []);
```

Lý do: Backend có `Post::with('comments')` nhưng comment cần `with('user', 'replies.user')` nested — tôi tách ra endpoint riêng `/posts/{id}/comments` để linh hoạt hơn. Tradeoff: 2 round-trips thay vì 1.

Tiếp theo: Phần 3 — Điểm cải thiện & Q&A Phỏng vấn